

Spring Security en Spring MVC

Sesión 2: Autorización, Roles, Authorities y
Persistencia en BD

Computación en Internet II — Universidad Icesi

Ruta de Aprendizaje — Sesión 2



01. Autorización a Fondo (AuthZ)

Modelo de control de acceso granular y SecurityContextHolder.



02. Roles vs Authorities

GrantedAuthority, prefijo ROLE_ y reglas hasRole vs hasAuthority.



03. Motor de Autenticación y BD

DaoAuthenticationProvider, UserDetailsService, BCrypt y RBAC.



04. Configuración y Vistas

SecurityFilterChain, Form Login custom y Thymeleaf Security.

01. Autorización (AuthZ) y Modelo de Seguridad

Control de acceso granular, principio de menor privilegio y contexto en memoria.



Control de Acceso



Menor Privilegio



Filtro AuthZ

Autorización: Control de Acceso Granular



De la Identidad a los Privilegios

En la Sesión 1 confirmamos *quién es el usuario* (Autenticación). Ahora debemos decidir: **¿A qué acciones y recursos tiene permiso?**



Principio de Menor Privilegio (PoLP)

Cada usuario debe contar únicamente con los permisos mínimos indispensables para realizar sus funciones operativas.



Evaluación Centralizada en Filtros

La autorización se resuelve antes de que el controlador comience a procesar la lógica de negocio.

El Modelo de Objetos de Seguridad

La Jerarquía del Contexto



SecurityContextHolder

Almacena el contexto de seguridad mediante `ThreadLocal` (aislado para cada hilo de petición HTTP).



SecurityContext

Envoltorio que contiene el objeto `Authentication` actualmente validado.



Authentication

Representa al usuario logueado con sus datos de identidad y lista de permisos concedidos.

Anatomía del Objeto Authentication

1. Principal

Representa al usuario (generalmente tu instancia personalizada de `UserDetails`).

2. Credentials

La contraseña o token ingresado (suele borrarse tras la autenticación por seguridad).

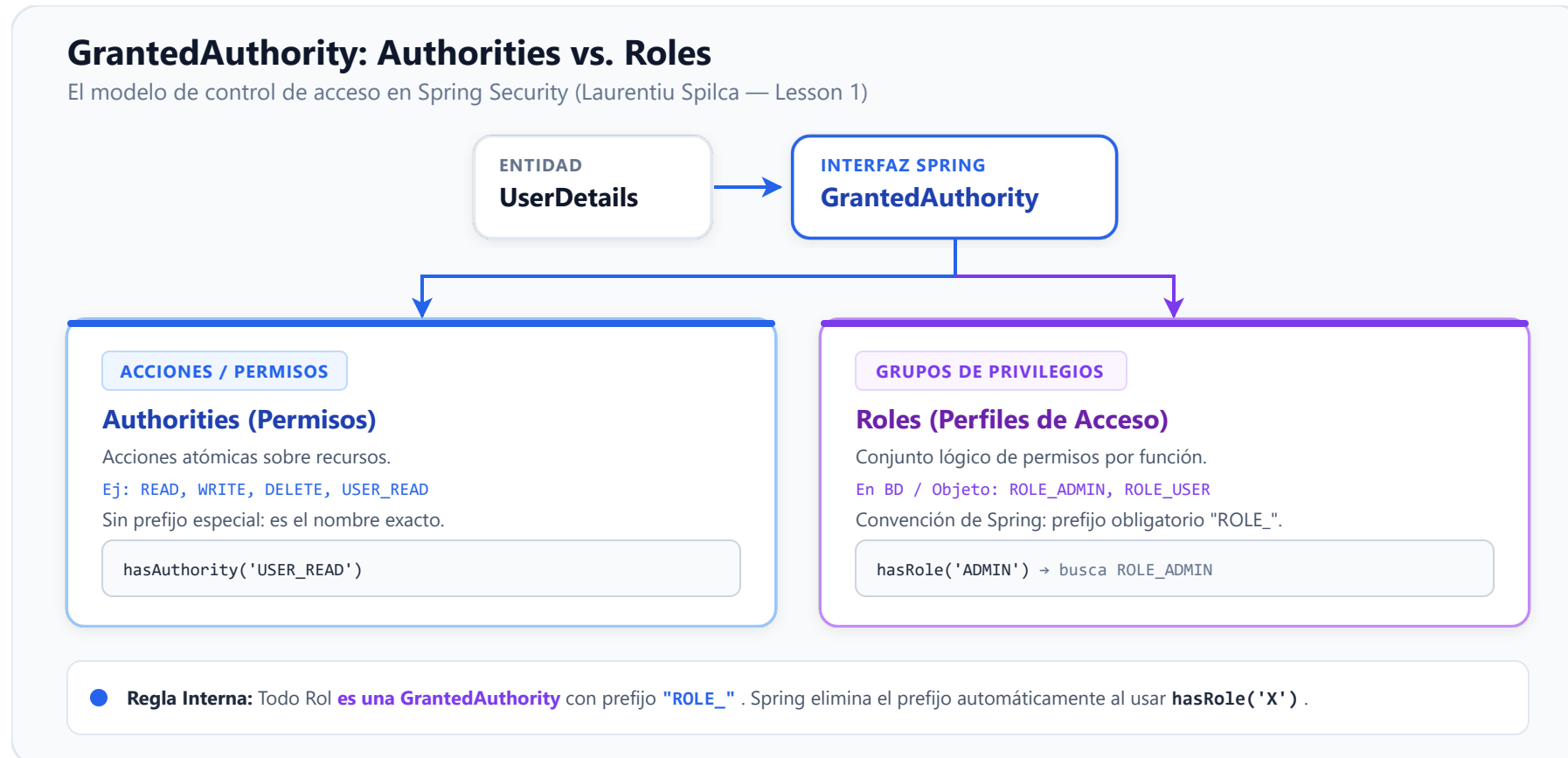
3. Authorities

Colección de `GrantedAuthority` (roles y permisos atómicos).

02. Roles vs Authorities: Diseño Limpio

GrantedAuthority, prefijo ROLE_, granularidad y reglas hasRole vs hasAuthority.

GrantedAuthority: Authorities vs. Roles



Reglas de Sintaxis: `hasRole()` vs `hasAuthority()`

HASROLE("ADMIN")

`hasRole("ADMIN")`

Spring Security agrega automáticamente el prefijo `ROLE_` internamente.

Busca que el usuario posea la authority `ROLE_ADMIN`.

HASAUTHORITY("ROLE_ADMIN")

`hasAuthority("ROLE_ADMIN")`

Busca coincidencia exacta de texto. No agrega ningún prefijo.

Exige especificar el nombre completo de la autoridad.

HASAUTHORITY("USER_CREATE")

`hasAuthority("USER_CREATE")`

Se utiliza para validar permisos granulares. Permite proteger endpoints basados en acciones específicas sin depender del rol global.

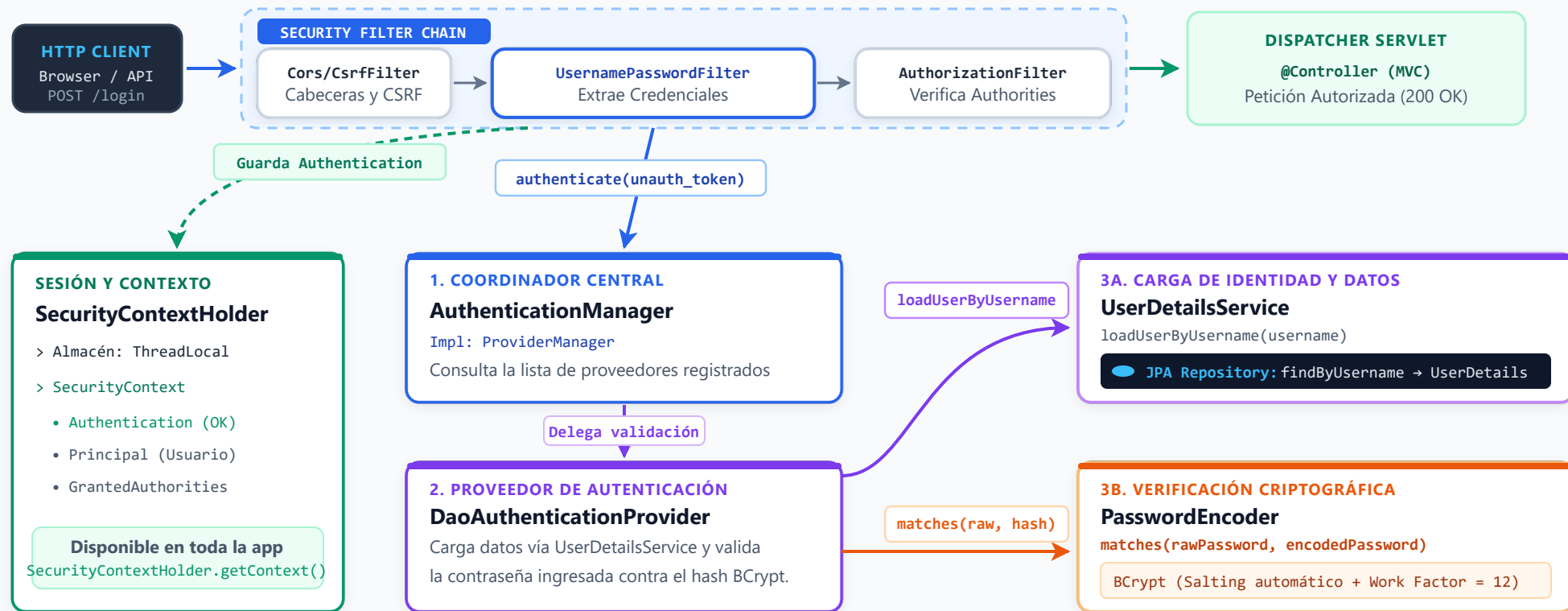
03. Motor de Autenticación y Persistencia JPA

DaoAuthenticationProvider, UserDetailsService, BCrypt y modelo relacional RBAC.

Arquitectura del Motor de Autenticación

Arquitectura Interna de Spring Security

Flujo de autenticación: SecurityFilterChain, AuthenticationManager, Providers y UserDetails



● Flujo: HTTP → AuthFilter → AuthManager → DaoAuthProvider → UserDetailsService (BD) → PasswordEncoder (BCrypt) → SecurityContext → DispatcherServlet (OK)

Contratos: UserDetailsService y UserDetails

Desacoplamiento en Spring Security

Spring Security no sabe cómo está construida tu tabla de usuarios ni qué campos tiene tu entidad JPA.



UserDetailsService

Interfaz funcional con un único método: `loadUserByUsername(String username)`.



UserDetails

Contrato que expone los datos requeridos por Spring: `getUsername()`, `getPassword()` y `getAuthorities()`.

```
@Service
public class CustomUserDetailsService
    implements UserDetailsService {

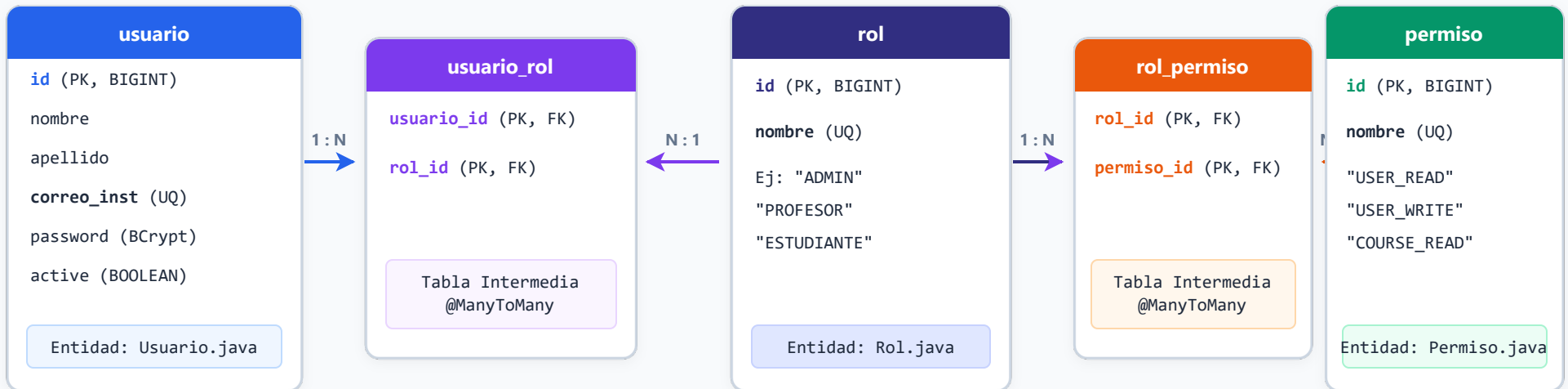
    @Autowired
    private UsuarioRepository usuarioRepository;

    @Override
    public UserDetails loadUserByUsername(String email)
        throws UsernameNotFoundException {
        Usuario usuario = usuarioRepository
            .findByCorreoInstitucional(email)
            .orElseThrow(() ->
                new UsernameNotFoundException("No existe:" + email));
        return new CustomUserDetails(usuario);
    }
}
```

Modelado Relacional RBAC en Base de Datos

Modelo Relacional de Control de Acceso (RBAC en BD)

Estructura de tablas en PostgreSQL/H2 (schema.sql / data.sql) mapeadas a JPA en el proyecto springboot



Mapeo a Spring Security en CustomUserDetails:

- **Roles:** Se mapean con el prefijo "ROLE_" + rol.getNombre() (ej. ROLE_ADMINISTRADOR) para hasRole('ADMINISTRADOR').
- **Authorities:** Se mapean directamente con permiso.getNombre() (ej. USER_READ) para hasAuthority('USER_READ').

Adaptadores: CustomUserDetails y Authority

```
public class SecurityAuthority
    implements GrantedAuthority {
    private final Permiso permiso;

    public SecurityAuthority(Permiso permiso) {
        this.permiso = permiso;
    }

    @Override
    public String getAuthority() {
        return permiso.getNombre();
    }
}
```

```
@AllArgsConstructor
public class CustomUserDetails implements UserDetails {
    private final Usuario usuario;

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        Set<GrantedAuthority> auths = new HashSet<>();
        for (Rol rol : usuario.getRoles()) {
            auths.add(new SimpleGrantedAuthority("ROLE_" + rol.getNombre()));
            for (Permiso p : rol.getPermisos()) {
                auths.add(new SimpleGrantedAuthority(p.getNombre()));
            }
        }
        return auths;
    }
}
```

Servicio: CustomUserDetailsService con JPA

Carga Transaccional de Datos

El servicio busca la entidad `Usuario` en BD por correo institucional y garantiza la carga de relaciones ManyToMany dentro de la transacción.



UsuarioRepository

Método derivado: `findByCorreoInstitucional(String email)` .



@Transactional(readOnly = true)

Mantiene la sesión de Hibernate abierta para inicializar roles y permisos sin `LazyInitializationException` .

```
@Service
@RequiredArgsConstructor
public class CustomUserDetailsService
    implements UserDetailsService {
    private final UsuarioRepository usuarioRepository;

    @Override
    @Transactional(readOnly = true)
    public UserDetails loadUserByUsername(String email)
        throws UsernameNotFoundException {
        Usuario u = usuarioRepository.findByCorreoInstitucional(email)
            .orElseThrow(() -> new UsernameNotFoundException("No existe:" + email));

        //Inicializar relaciones Lazy
        u.getRoles().forEach(r -> r.getPermisos().size());
        return new CustomUserDetails(u);
    }
}
```

04. Configuración Declarativa con SecurityFilterChain

SecurityFilterChain DSL, formulario de login propio, logout y tags sec:authorize.

Reglas Declarativas de Autorización de Rutas

Reglas Declarativas en HttpSecurity

En Spring Boot 3+, la configuración se realiza definiendo un Bean de tipo `SecurityFilterChain` mediante expresiones Lambda.



requestMatchers + permitAll()

Rutas públicas para login, recursos estáticos (/css, /js) o registro.



requestMatchers + hasRole()

Rutas protegidas por perfil o privilegio específico.



anyRequest().authenticated()

Cualquier otra ruta exige obligatoriamente sesión activa.

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http)
    throws Exception {
    return http
        .authorizeHttpRequests(authz -> authz
            .requestMatchers("/css/**", "/js/**", "/images/**").permitAll()
            .requestMatchers("/roles-mvc/**", "/permisos-mvc/**").hasRole("ADMINISTRADOR")
            .requestMatchers("/profesores-mvc/**").hasAnyRole("ADMINISTRADOR", "PROFESOR")
            .requestMatchers("/cursos-mvc/**", "/matriculas-mvc/**").authenticated()
            .anyRequest().authenticated()
        )
        .formLogin(form -> form.loginPage("/login").defaultSuccessUrl("/usuarios", true))
        .logout(logout -> logout.logoutUrl("/logout").logoutSuccessUrl("/login?logout"))
        .build();
}
```



Vista Custom



POST Intercept



Success URL

Formulario de Login Personalizado



loginPage("/login")

Ruta del controlador Spring MVC que renderiza la vista Thymeleaf.



defaultSuccessUrl("/usuarios", true)

Página de redirección inmediata tras autenticación exitosa.



usernameParameter("username")

Nombre del input HTML para el identificador (correo institucional).



failureUrl("/login?error")

Redirección con parámetro de error si las credenciales fallan.

```
.formLogin(form -> form
    .loginPage("/login")
    .loginProcessingUrl("/login")
    .defaultSuccessUrl("/usuarios", true)
    .failureUrl("/login?error")
    .usernameParameter("username")
    .passwordParameter("password")
    .permitAll()
)
```

Gestión Segura de Logout

1. ENDPOINT DE SALIDA

logoutUrl("/logout")

Ruta interceptada por `LogoutFilter` (por defecto POST para proteger contra ataques CSRF).

2. INVALIDACIÓN EN SERVIDOR

Invalidación de Sesión

Destruye la `HttpSession` en la memoria RAM del servidor y limpia el `SecurityContextHolder`.

3. ELIMINACIÓN DE COOKIE

Limpieza de Cookies

Envía un header para eliminar la cookie `JSESSIONID` en el navegador y redirige a `/login?logout`.

Control de Vistas con Thymeleaf Extras

Seguridad en Interfaz de Usuario

Con el dialecto `thymeleaf-extras-springsecurity6`, la interfaz se adapta dinámicamente al perfil del usuario autenticado.



`sec:authorize="hasRole('ADMINISTRADOR')"`

Renderiza botones o menús administrativos únicamente si el usuario posee dicho rol.



`sec:authentication="name"`

Imprime el correo o nombre del usuario autenticado en la barra superior.

```
<!-- Barra de Navegación Thymeleaf -->
<div class="navbar">
  <span>Conectado:
    <strong sec:authentication="name">usuario</strong>
  </span>

  <!-- Visible solo para Administradores -->
  <a th:href="@{/roles-mvc}"
    sec:authorize="hasRole('ADMINISTRADOR')">
    Gestión de Roles
  </a>

  <form th:action="@{/logout}" method="post">
    <button type="submit">Cerrar Sesión</button>
  </form>
</div>
```

Errores Frecuentes y Buenas Prácticas

LAZY LOADING EN JPA

LazyInitializationException

Ocurre al acceder a `usuario.getRoles()` fuera de sesión Hibernate. *Solución: Usar `@Transactional(readOnly = true)` e inicializar colecciones.*

PREFIJO DE ROLES

Confusión con Prefijo ROLE_

`hasRole('ADMIN')` busca `ROLE_ADMIN`.
No ingresar `hasRole('ROLE_ADMIN')` porque buscaría `ROLE_ROLE_ADMIN`.

TOKENS CSRF

Olvido de Token CSRF

Formularios HTML puros sin Thymeleaf (`th:action`) fallarán con 403 Forbidden por omisión del campo `_csrf` oculto.

HASHING DE BD

Contraseñas no Hasheadas

Insertar contraseñas en texto plano en la BD causará fallos en `BCryptPasswordEncoder.matches()`. Usar script con hash generado.

Live Demo 3: Prueba Integral de Acceso

Actividad en Vivo: Verificación Integral RBAC



1. Inspeccionar Usuario en BD

Verificar que `data.sql` contiene hashes que inician con `$2a$10$...` (BCrypt).



2. Login con Administrador

Ingresa con `admin@icesi.edu.co` y comprobar acceso total a `/roles-mvc`.



3. Probar Aislamiento de Estudiante

Ingresa con `apaez@icesi.edu.co` y verificar código `403 Forbidden` al intentar acceder a rutas administrativas.



4. Verificar Barra Condicional

Comprobar que el botón de roles desaparece automáticamente para estudiantes en Thymeleaf.

// Registro SQL en data.sql

```
INSERT INTO usuario (nombre, correo_institucional, password, active)
```

```
VALUES (
```

```
  'Super Admin',
```

```
  'admin@icesi.edu.co',
```

```
  '$2a$10$wK5W3yC3yF15D1nB.vN7teI0wL/y6V5t1oB97GgD4aAefXkK/Zl9G',
```

```
  TRUE
```

```
);
```

Laboratorio Práctico

Implementando Spring Security en el Proyecto
MVC